

DS_Project_1 REPORT



학번 : 2022202109

이름 : 이호정

► Instruction

이 프로그램은 자막 파일을 관리하고 여러 명령어에 해당하는 기능을 활용해 자막 파일을 탐색하는 일을 하도록 한다. 이런 프로그램을 c++ 로 만드는 것이 이번 과제 목표이다. 주어진 스켈레톤 코드와 소개 pdf 를 보면 이 프로그램은 자막의 시간을 기준으로 이진 탐색 트리 BST 를 사용하여 자막 데이터를 정렬하고 관리하고, 큐를 이용해 자막을 임시로 저장 및 처리한다. 추가적으로, 섹션 관리 기능을 통해 시간 범위대 별로 자막을 구분하고 관리할 수 있다. 프로그램에는 여러 기능이 있는데 메인이 되는 주요 기능 몇 개를 소개하자면 아래와 같다.

- LOAD

자막 데이터를 텍스트 파일로부터 읽어 들여 큐에 저장한 후 출력하도록 한다. 자막의 시간과 자막의 내용을 구분하여 저장하고, 큐는 먼저 들어온 데이터를 먼저 처리하는 선입선출 (FIFO, first in first out) 방식의 자료구조를 의미한다. 파일이 제대로 열리지 않는다면 오류 메시지를 출력하고 프로그램을 종료하도록 한다.

- QPOP

큐에 저장된 데이터를 모두 꺼내 BST 에 삽입한다. Tree 에는 자막시간을 토대로 정렬되어 저장된다. 이렇게 이진 탐색 트리를 사용함으로써 자막 데이터의 검색, 추가, 삭제 등을 빠르게 처리 할 수 있게 된다.

- PRINT

트리에 정렬되어 저장된 데이터들을 출력하는 기능을 수행한다. 지금까지 저장된 데이터들을 시간순으로 출력할 수 있으며, 특정 구간에 해당하는 자막 섹션을 출력할 수도 있다. 이 기능은 이전에 섹션을 선언했을 때만 가능하다. 트리에 저장된 자막을 중위 순회하며 출력하기 때문에 시간순으로 자막 데이터들을 볼 수 있다.

- SECTION

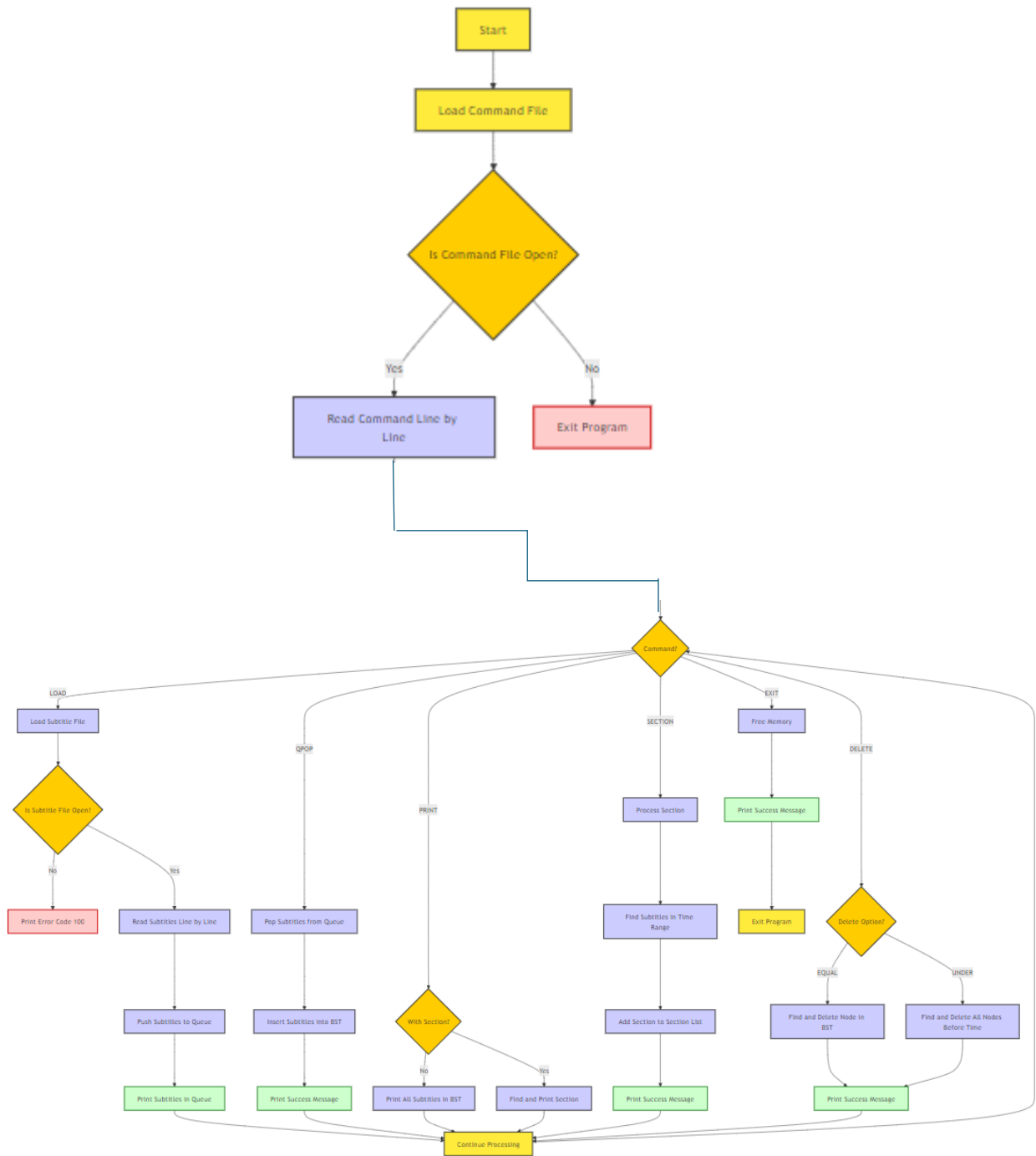
특정 시간 범위에 해당하는 자막을 찾아 리스트로 연결하고, 이를 섹션 할당하고 관리할 수 있다. 그렇게 섹션을 할당하면 특정 시간 구간의 자막들만 빠르게 조회 하거나 처리할 수 있다. 그리고 필요한 시간대의 데이터들을 묶어서 용이하게 관리할 수 있다.

-DELETE

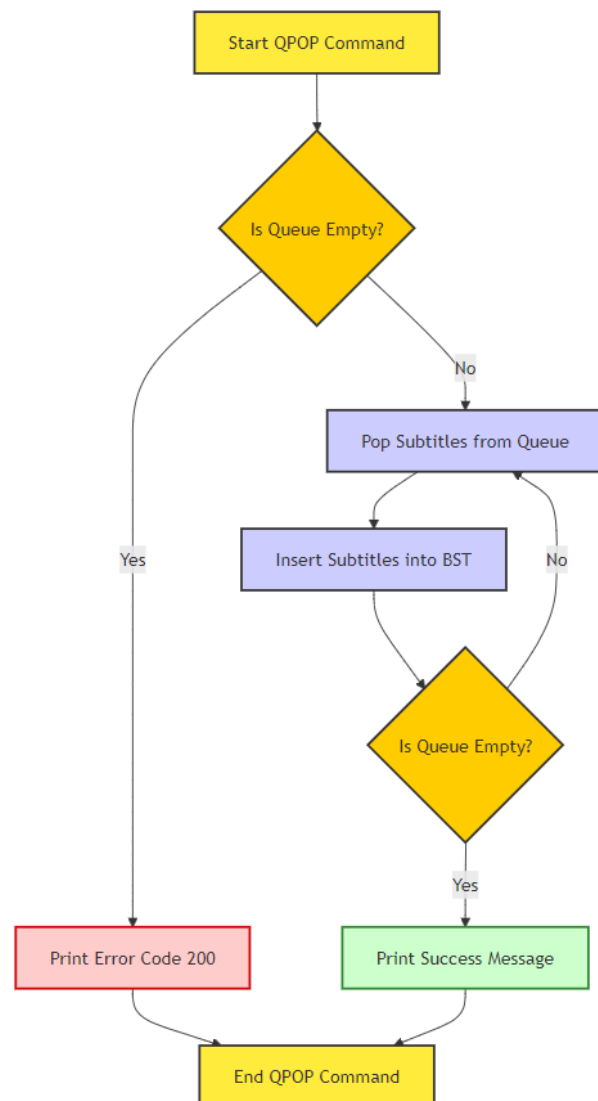
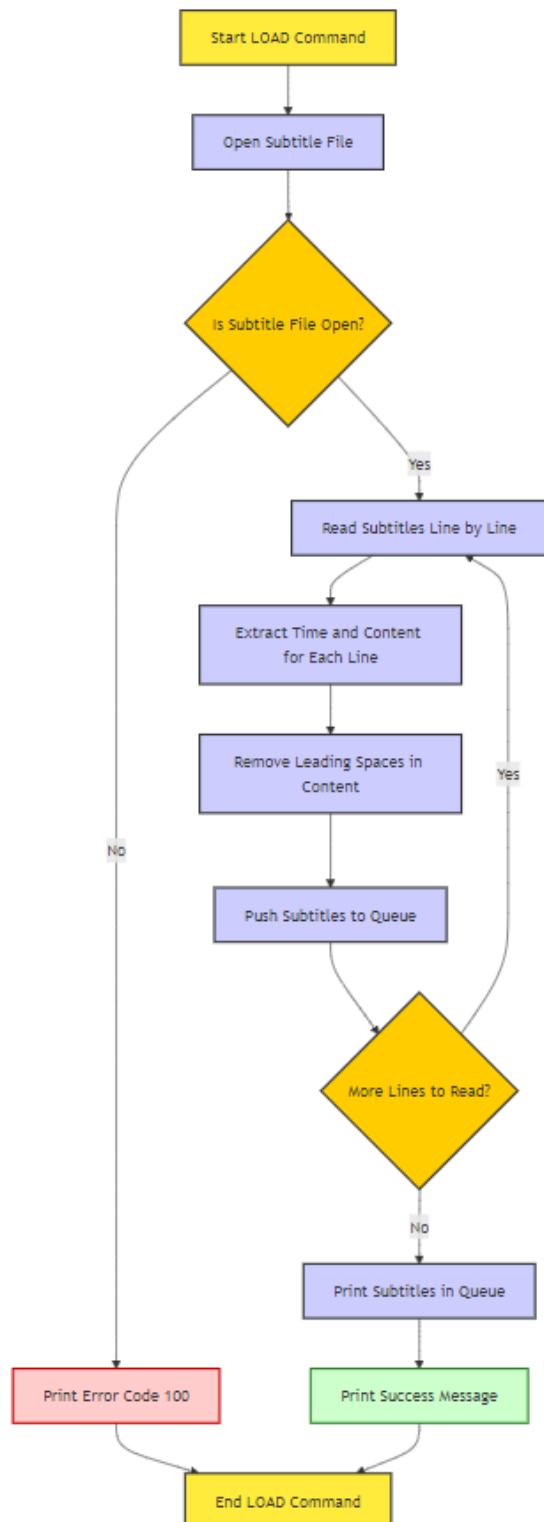
특정 자막을 시간 기준으로 삭제하거나, 지정한 시간보다 이전에 해당하는 모든 자막을 삭제하는 기능이다. DELETE EQUAL 은 특정 시간대의 자막 데이터 하나만 삭제하고, DELETE UNDER 는 입력 시간 보다 이전의 자막들을 모두 삭제한다.

► Flowchart

-> 전체적인 flowchart, 주요 function flowchart 로 나누어 표현



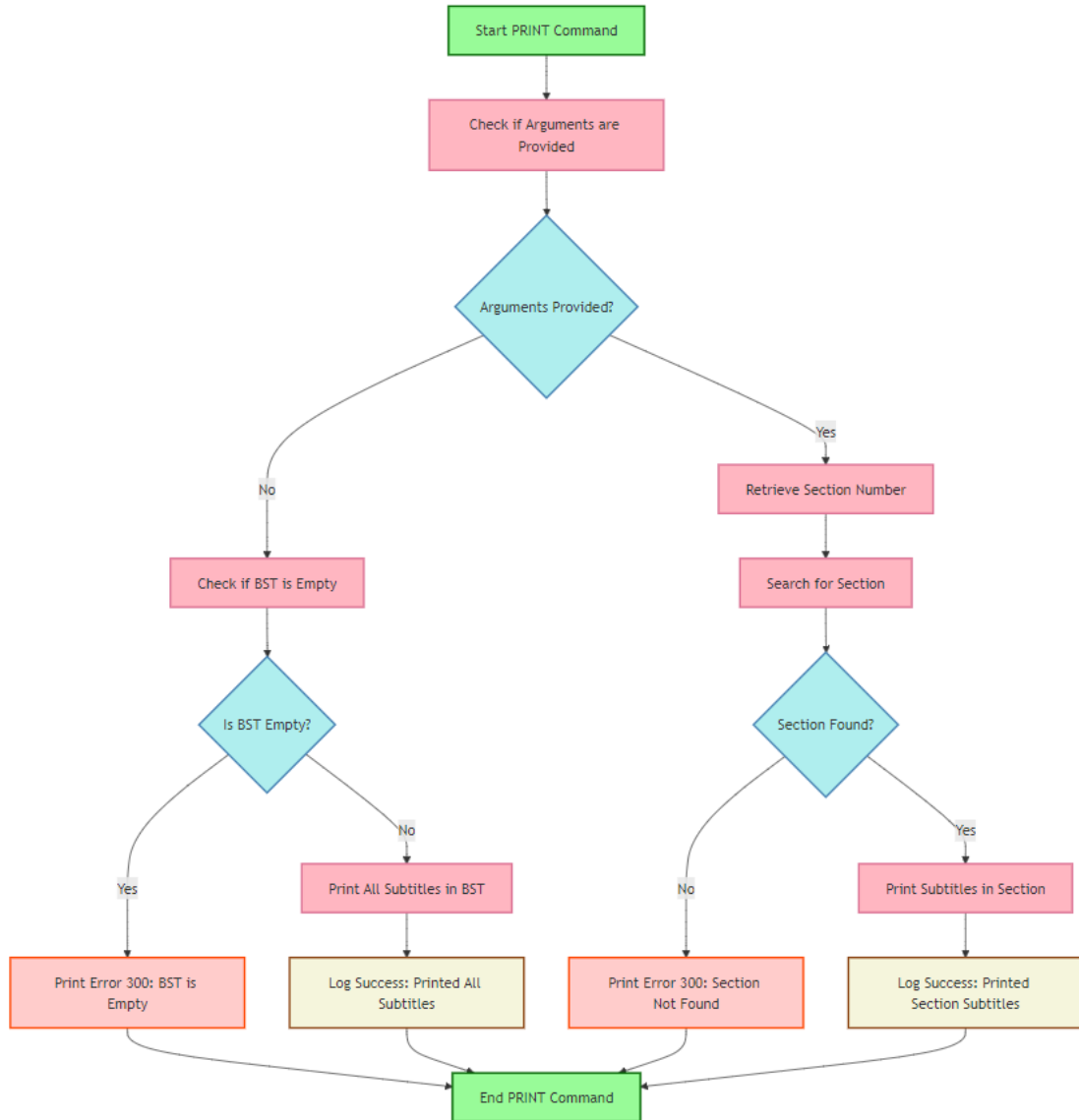
► LOAD flowchart / QPOP flowchart



► delete flowchart



► PRINT flowchart



► Algorithm

→ 이 프로그램에서 사용된 알고리즘에 대해 설명하고, 어떤 부분이 그 알고리즘인지 자세히 작성하였다.

1. Queue_(LOAD/QPOP 에서 사용)

이 프로그램에서는 자막 파일의 데이터들을 큐에 담아서 관리한다. 이 과정에서 데이터를 큐에 담아서 관리하려면 이러한 동작을 하는 자료구조를 만들어 사용해야 했다. 큐는 자막을 읽어들이고 이를 이용해 QPOP 명령을 통해 BST 로 옮길 때 필요하다. 큐의 특징으로는 선입선출, 즉 먼저 들어온 데이터가 먼저 처리된다는 특징이 있다. 큐에는 기능의 알고리즘이 존재하는데, 먼저 자막 파일을 한 줄 씩 읽고, 자막의 시간과 내용을 끝 부분에 삽입한다. 큐가 가득 찼는지 확인하는 isEmpty/isFull 함수가 포함되어있다. 만약 큐가 가득 찼는데 데이터를 삽입하려하거나 큐가 비었는데 데이터를 pop 하려고 하면 프로그램이 종료되도록 설계하였다.

메인이라고 할 수 있는 큐에 새로운 노드를 삽입하는 알고리즘에 대해 간략히 설명해보면 아래와 같다. 먼저 isEmpty 함수로 큐가 비었는지 확인하고, 큐가 비었으면 삽입하는 노드가 첫 번째 노드가 될 테니 front 포인터와 rear 포인터 모두 newNode 를 가리키도록 한다. 그리고 큐가 비지 않았다면 마지막 노드 뒤에 뉴노드를 입력해야 하는 것이니 기존 rear 포인터가 가리키고 있는 노드의 다음 노드를 뉴노드를 가리키도록 set 함수를 활용해 설정해주고, rear를 새노드로 이동시킴으로 큐에 새로운 노드를 삽입 할 수 있다.

그리고 큐 내의 pop 을 설명하면 다음과 같다. 먼저 이 프로그램에 큐의 pop 이 중요한 이유는 나중에 이를 이용해 bst 에 값을 넣을 때 중요하기 때문이다. (물론 front 함수도) 먼저 큐가 비어있을 경우 pop 을 시도하면 프로그램을 종료하도록 하고, 아닌 경우는 pop 로직을 실행한다. 먼저 temp 노드를 만들어 front 노드를 임시 저장하도록 한다. 그런 후 front 를 다음 노드로 이동시킨다. 그것은 getNext 함수를 사용하여 지정한다. 그리고 만약 큐가 비었다면, 즉 front 가 null 이 되었다면 rear 도 null 로 지정해 준다. 이것은 아마 pop 한 노드가 큐의 마지막 노드인 경우일 것이다. 그리고 temp를 메모리 해제 시키고 큐의 사이즈를 줄임으로써, pop 를 하는 로직이 완성된다.

그리고 아까 잠시 언급한 front 함수에 대해 간략히 설명하자면 이는 큐의 가장 앞에 있는 노드를 반환 하지만 삭제하지는 않는 함수이다. 로직은 간단하다. 큐가 비었는지 확인하고, 비지 않았다면 return front 를 해주어 노드를 반환한다. 그러나 이 노드는 큐에 아직 남아있는 것이다. 삭제가 아니다.

2. BST _ (QPOP/PRINT/DELETE/SECTION)

사실 이 알고리즘은 특정 어느 부분에서만 사용되었다기엔 모든 명령어에 관여한다. 자막 데이터들을 시간 기준으로 정렬하고 관리하면서, 순회할 수 있는 이유는 모두 이 BST, 이진 탐색 트리에 저장하고 관리했기 때문이다. BST 알고리즘에 대해 설명하자면, 먼저 큐에 저장된 데이터들이 QPOP 명령어를 통해 BST에 하나씩 저장되게 된다. 그리고 이를 시간 순으로 삽입하는데, 이진 탐색 트리에 새로운 자막 데이터가 POP 되어 들어 오면 이 노드의 위치를 정해줄 것이다. 새로운 노드의 자막 시간이 현재 노드의 시간보다 작으면 왼쪽 서브트리로, 크면 오른쪽 서브트리로 이동하면서 재귀적으로 삽입하는 알고리즘이다.

그리고 이렇게 모든 삽입이 끝나고, 특정 노드를 찾기 위한 Search 함수에는 검색 알고리즘이 포함되어있다. Section 명령어 등, 특정 노드를 찾아야 한 경우에는 시간에 따라 트리를 탐색하여 찾고자 하는 시간 데이터에 해당하는 자막 데이터를 찾는 알고리즘이다. 그리고 이렇게 찾은 데이터를 삭제하는 알고리즘도 포함 되어있다. 삭제한 노드를 찾은 후, 해당 노드를 bst 에서 제거하는 것 또한 큰 일이었다. 먼저 삭제할 노드를 찾으면, 그 노드가 자식 노드를 가지고 있는지 아닌지 여부에 따라서 설계를 달리한다. 자식 노드가 없을 경우에는 부모 노드와 자식 노드 사이의 연결을 단순히 끊으면 된다. 그 방법으로는 부모노드와 자식노드가 연결되어있는 포인터의 연결을 끊어줌으로써 해당 노드를 bst 에서 삭제 되도록 한다. 그리고 자식노드가 하나인 경우, 두개인 경우도 나누어 생각해 주었다. 먼저 자식 노드가 하나인 경우, 그 자식 노드를 부모 노드와 연결한 후 삭제한다. 그리고 자식노드가 두개인 경우에는 오른쪽 서브 트리에서 최소값을 찾아 그 노드를 삭제하고 삭제하고자 했던 노드를 대신하도록 한다. 그러면 시간 순서에도 맞고, 중복되는 노드도 없이 내가 삭제하고자 하는 노드를 올바르게 삭제할 수 있다. 이 부분을 코딩하면서도 많은 실수가 있었는데, 먼저 최소값을 가진 노드의 자막 정보를 삭제하고자 하는 노드의 데이터로 세팅해주면서 삭제하고자 한 노드의 자막 정보를 지워버린다. 이렇게 한 후 최소값의 노드를 오른쪽 서브트리에서 삭제하고, 순서에 알맞은 bst 가 유지 된다. 그리고 마지막으로 중위 순회 알고리즘이 있다. 이진 탐색 트리를 중위 순회하여 시간 순서대로 자막을 출력하는 알고리즘으로써, 왼쪽 서브 트리부터 출력을 하고, 현재 노드를 출력한 후, 다시 오른쪽 서브 트리를 출력하는 로직이다.

3. LIST

리스트는 섹션 개념을 구성하기 위해서 사용된다. 사실 SECTION 명령어가 없다면 BST 까지만 있어도 관리가 가능 했겠지만, 섹션 넘버와 구간을 설정하고 관리하려면 LIST 형태로 만들어야 한다. 그 데이터들을 연결 리스트 형태로 관리하게 한다. 이를 통해 특정 시간 범위에 있는 자막들을 손쉽게 그룹화 할 수 있다. 노드 삽입할 때, 특정 섹션 번호와 함께 해당하는 자막 리스트를 추가하는 알고리즘이 필요하다. 이 리스트는 연결리스트 형태로 섹션 번호에 따라 자막들을 저장한다. 이중 링크드 리스트 형태로 첫 번째 리스트에는 섹션 넘버가, 두 번째 리스트에는 자막 데이터 노드가 저장되어있다. 그리고 단순히 저장 뿐만 아니라 특정 섹션 번호를 기준으로 섹션 리스트에서 해당 섹션을 검색하는 알고리즘 또한 필요하다. 섹션 리스트를 순차적으로 검색하여 해당 번호를 가진 섹션을 찾도록 하였다.

▶ Result Screen

➔ 모든 명령어에 대한 결과 화면과 동작 설명

1. 과제 예제 command 실행 결과

```
≡ command.txt
1  LOAD
2  QPOP
3  PRINT
4  SECTION 3 00:52:10 00:52:30
5  SECTION 2 00:53:50 00:54:23
6  PRINT 3
7  PRINT 2
8  DELETE EQUAL 00:52:36
9  PRINT
10 DELETE UNDER 00:52:23
11 PRINT
12 PRINT 3
13 EXIT
```

command.txt

```
===== LOAD =====
01:03:45 - You're not listening to me!
00:52:36 - You're welcome.
00:54:16 - Hey, you! Where's the other mother?
00:54:18 - I wanna go home.
00:53:50 - Mom! Dad!
00:53:56 - Oh, God. I'm still here?
00:54:19 - All will be swell, soon as Mother's refreshed.
00:54:23 - Her strength is our strength.
00:52:21 - Bed? Before dinner?
00:52:20 - Right now!
00:52:19 - I'm going to bed.
00:21:37 - See you soon.
00:52:23 - I'm really, really tired. Yeah.
01:09:34 - Ok.
01:09:24 - Challenge her, then.
01:09:29 - She's got a thing for games.
01:09:26 - She may not play fair, but she won't refuse.
01:09:17 - I have to go back. They are my parents.
01:11:36 - A finding things game.
01:11:27 - Everybody likes games.
01:11:51 - What if you don't find them?
01:11:59 - And I'll let you sew buttons into my eyes.
01:11:54 - If I lose, I'll stay here with you forever
01:11:31 - What kind of game would it be?
01:11:38 - And what is it you'd be finding?
01:11:41 - My real parents. And... the eyes of the children.
01:11:26 - I know you like them.
01:11:22 - Why don't we play a game?
=====
```

LOAD 명령어 실행결과 :

subtitle.txt 의 데이터들이 올바르게 LOAD 된 것을 확인 할 수 있다. 아직 정렬이 된 것은 아니게 맞다.

```
===== QPOP =====  
Success  
=====
```

QPOP 명령어 실행결과 : 올바르게 실행되고, 결과를 SUCCESS 라고 출력하는 것을 확인할 수 있다.

```
===== PRINT =====  
00:21:37 - See you soon.  
00:52:19 - I'm going to bed.  
00:52:20 - Right now!  
00:52:21 - Bed? Before dinner?  
00:52:23 - I'm really, really tired. Yeah.  
00:52:36 - You're welcome.  
00:53:50 - Mom! Dad!  
00:53:56 - Oh, God. I'm still here?  
00:54:16 - Hey, you! Where's the other mother?  
00:54:18 - I wanna go home.  
00:54:19 - All will be swell, soon as Mother's refreshed.  
00:54:23 - Her strength is our strength.  
01:03:45 - You're not listening to me!  
01:09:17 - I have to go back. They are my parents.  
01:09:24 - Challenge her, then.  
01:09:26 - She may not play fair, but she won't refuse.  
01:09:29 - She's got a thing for games.  
01:09:34 - Ok.  
01:11:22 - Why don't we play a game?  
01:11:26 - I know you like them.  
01:11:27 - Everybody likes games.  
01:11:31 - What kind of game would it be?  
01:11:36 - A finding things game.  
01:11:38 - And what is it you'd be finding?  
01:11:41 - My real parents. And... the eyes of the children.  
01:11:51 - What if you don't find them?  
01:11:54 - If I lose, I'll stay here with you forever  
01:11:59 - And I'll let you sew buttons into my eyes.  
=====
```

PRINT 명령어 실행 결과

QPOP이 성공적으로 실행되었기 때문에 자막 데이터 정보들이 시간순으로 정렬 정렬 출력되는 것을 확인 할 수 있다.

```
===== SECTION =====  
Success  
=====
```



```
===== SECTION =====  
Success  
=====
```

SECTION 명령어 실행결과

SECTION 3 00:52:10 00:52:30

SECTION 2 00:53:50 00:54:23

위와 같은 두개의 섹션을 만드는것에 성공한 후, 성공메세지를 출력한 것을 확인할 수 있다.

```

===== PRINT =====
Section 3
00:52:19 - I'm going to bed.
00:52:20 - Right now!
00:52:21 - Bed? Before dinner?
00:52:23 - I'm really, really tired. Yeah.
=====
===== PRINT =====
Section 2
00:53:50 - Mom! Dad!
00:53:56 - Oh, God. I'm still here?
00:54:16 - Hey, you! Where's the other mother?
00:54:18 - I wanna go home.
00:54:19 - All will be swell, soon as Mother's refreshed.
00:54:23 - Her strength is our strength.
=====

```

PRINT N 실행결과

PRINT N(숫자) 명령어를 입력하며 그 숫자에 해당하는 섹션의 범위내 데이터들이 올바르게 출력되는 것을 확인 할 수 있다. Command 텍스트 파일을 확인해 보면

SECTION 3 00:52:10 00:52:30 와 SECTION 2 00:53:50 00:54:23 로 해당 시간이 올바르게 섹션 설정 된 것을 확인 할 수 있다. 또 PRINT 명령어 뿐만 아니라 PRINT N 명령어 또한 올바르게 실행되는 것을 확인 할 수 있다.

```

===== DELETE =====
Success
=====

===== PRINT =====
00:21:37 - See you soon.
00:52:19 - I'm going to bed.
00:52:20 - Right now!
00:52:21 - Bed? Before dinner?
00:52:23 - I'm really, really tired. Yeah.
00:53:50 - Mom! Dad!
00:53:56 - Oh, God. I'm still here?
00:54:16 - Hey, you! Where's the other mother?
00:54:18 - I wanna go home.
00:54:19 - All will be swell, soon as Mother's refreshed.
00:54:23 - Her strength is our strength.
01:03:45 - You're not listening to me!
01:09:17 - I have to go back. They are my parents.
01:09:24 - Challenge her, then.
01:09:26 - She may not play fair, but she won't refuse.
01:09:29 - She's got a thing for games.
01:09:34 - Ok.
01:11:22 - Why don't we play a game?
01:11:26 - I know you like them.
01:11:27 - Everybody likes games.
01:11:31 - What kind of game would it be?
01:11:36 - A finding things game.
01:11:38 - And what is it you'd be finding?
01:11:41 - My real parents. And... the eyes of the children.
01:11:51 - What if you don't find them?
01:11:54 - If I lose, I'll stay here with you forever
01:11:59 - And I'll let you sew buttons into my eyes.
=====

```

DELETE EQUAL 명령어 실행 결과

COMMAND 텍스트 파일을 확인해 보면 첫 번째 DELETE 명령어로

DELETE EQUAL 00:52:36 를 입력한 것을 확인 할 수 있다. 그 결과 올바르게 DELETE 를 수행하고, 성공메세지를 띄운 후, PRINT 를 하면 입력한 노드와 같은 노드인 00:52:36 부분이 사라진 것을 확인 할 수 있다. 이로써 DELETE EQUAL 명령어 또한 올바르게 동작되는것을 확인 할 수 있다.

```

===== DELETE =====
Success
=====

===== PRINT =====
00:52:23 - I'm really, really tired. Yeah.
00:53:50 - Mom! Dad!
00:53:56 - Oh, God. I'm still here?
00:54:16 - Hey, you! Where's the other mother?
00:54:18 - I wanna go home.
00:54:19 - All will be swell, soon as Mother's refreshed.
00:54:23 - Her strength is our strength.
01:03:45 - You're not listening to me!
01:09:17 - I have to go back. They are my parents.
01:09:24 - Challenge her, then.
01:09:26 - She may not play fair, but she won't refuse.
01:09:29 - She's got a thing for games.
01:09:34 - Ok.
01:11:22 - Why don't we play a game?
01:11:26 - I know you like them.
01:11:27 - Everybody likes games.
01:11:31 - What kind of game would it be?
01:11:36 - A finding things game.
01:11:38 - And what is it you'd be finding?
01:11:41 - My real parents. And... the eyes of the children.
01:11:51 - What if you don't find them?
01:11:54 - If I lose, I'll stay here with you forever
01:11:59 - And I'll let you sew buttons into my eyes.
=====

```

DELETE UNDER 명령어 실행 결과

이 명령어는 인자로 입력받은 시간대 이전의 노드를 전부 삭제하는 명령어 이다.

DELETE UNDER 00:52:23 을 값으로 입력받았고, delete 실행 후, 해당 노드보다 작은 시간대의 노드들은 모두 삭제 된 것을 확인할 수 있다. '보다 작은' 이 조건이기 때문에 같은 노드는 삭제 되지 않았음을 확인할 수 있다.

```

===== PRINT =====
Section 3
00:52:19 - I'm going to bed.
00:52:20 - Right now!
00:52:21 - Bed? Before dinner?
00:52:23 - I'm really, really tired. Yeah.
=====

```

PRINT N 명령어 실행 결과

노드를 삭제한 이후에 SECTION3 에 담긴 값을 다시 출력하는 명령어를 실행 한 결과이다. 이때 주목해야 할 부분은 DELETE UNDER 로 BST 에서는 SECTION3 내에 속한 노드가 삭제 되어야 했음에도 LIST 이기 때문에 삭제 되지 않는다는 것을 확인 할 수 있다.

```

===== EXIT =====
Success
=====

```

EXIT 명령어 수행결과

EXIT 명령어가 올바르게 수행되어 성공메세지를 출력하고 프로그램을 종료한 것을 확인할 수 있다.

➔ 테스트 코드를 만들어 여러 예외 상황을 테스트 해보고 코드가 올바르게, 결과가 정상 적으로 출력되는지 확인하였다.

1. DELETE EQUAL 명령어에서 없는 시간의 노드를 입력으로 주었을 때

<pre>PRINT 2 DELETE EQUAL 08:52:36 PRINT DELETE UNDER 08:52:36</pre>	<pre>===== ERROR ===== 500 =====</pre>
--	--

➔ 위의 예시와 같이 없는 시간의 노드를 인자로 주면 DELETE 의 에러 코드인 500 을 출력하는 것을 확인 할 수 있다.

2. DELETE UNDER + 큰 시간대의 노드를 입력으로 주었을 때

<pre>DELETE UNDER 12:52:23 PRINT PRINT 2</pre>	<pre>===== DELETE ===== Success ===== ===== ERROR ===== 300 =====</pre>
--	--

➔ 위와 같이 최대 시간 이상의 시간을 삭제 기준으로 넣어주면 BST 의 모든 노드가 삭제 되어, 그 이후 PRINT 명령어를 실행했을 때 에러가 발생하는 것을 확인 할 수 있다. 그리고 그 PRINT 명령어의 에러코드인 300이 올바르게 출력되는 것을 확인 할 수 있다.

3. SECTION 에 유효하지 않은 범위의 시간대를 입력으로 주었을 때

<pre>SECTION 2 24:53:50 25:54:23 PRINT 2</pre>	<pre>===== ERROR ===== 400 ===== ===== ERROR ===== 300 =====</pre>
--	---

➔ section 명령어와 유효하지 않은 시간 범위를 넣었을 때, 에러 코드 400이 출력 되는지 확인하였다. SECTION 명령어 에러코드 400과 PRINT 명령어 에러코드 300 모두 올바르게 출력되는 것을 확인 할 수 있다.

4. DELETE EQUAL + 존재하지 않는 노드를 입력했을 경우

```
DELETE EQUAL 12:12:12  
PRINT
```

```
===== ERROR =====  
500  
=====
```

- 위와 같이 delete equal 명령어와 존재하지 않는 시간의 노드를 입력했을 때 올바르게 에러코드가 뜨는지 확인하였다.

5. 존재하지 않는 명령어를 입력했을 경우

```
ENTP
```

```
===== ERROR =====  
1000  
=====
```

- 위와 같이 존재하지 않는 명령어를 입력했을 때, 명령어 에러코드인 1000이 출력되는 것을 확인 할 수 있다.

6. 명령어의 인자가 부족하거나 초과할 경우

```
DELETE EQUAL  
PRINT  
DELETE UNDER 00:52:23 00:11:22  
PRINT
```

```
===== ERROR =====  
500  
=====
```



```
===== ERROR =====  
500  
=====
```

- 위와 같이 DELETE EQUAL 명령어를 입력하고 뒤에 필요한 인자를 주지 않거나, 혹은 UNDER 명령어에 시간노드를 2개 입력하는 등, 인자를 많거나 적게 줬을 경우에 에러코드가 뜨는 것을 확인 할 수 있다. 이때 에러코드는 잘못된 명령어 에러코드인 1000이 아니라 해당 명령어의 오류코드가 올바르게 출력되는 것을 확인 할 수 있다.
- 이 설명에 대한 예시는 아래와 같은 입력을 넣고 결과 화면을 확인해 보았다.

LOAD A	===== ERROR =====
QPOP A	100
PRINT A	=====
SECTION A A A A	===== ERROR =====
	200
	=====
	===== ERROR =====
	300
	=====
	===== ERROR =====
	400
	=====

➔ 위에서 설명한 바와 같이 이렇게 뒤의 인자의 개수가 잘못 되어있는 경우에는 잘못된 명령어 에러 코드인 1000이 아닌 각각의 해당 명령어의 에러코드가 올라올바 출력되는 것을 확인하여 보았다.

► Consideration

- ➔ 이 프로그램을 설계하면서 데이터 구조에 대해 보다 깊게 알게 되었다. 객체 지향 프로그래밍 수업 때 배웠던 class 를 기본 베이스로 큐, BST, List 등 객체 지향 프로그래밍 시간에 맛봤던 알고리즘들을 응용하면서 그에 대한 이해와 활용도 능력치가 많이 상승했다고 느꼈다. 그리고 이렇게 직접 코딩하면서 내가 class 의 기본 개념에 대해서 부족한 점이 있었다는 것들도 느끼게 되었다.
- ➔ 그렇다면 내가 코드를 설계하면서 막혔던 부분들에 대해서 이야기 해보겠다. 코드를 짜면서 막혔던 부분이 여러 번 있었는데, 그로 인해 새롭게 알게된 사실들이 많이 있었다. 그리고 이렇게 몸소 겪게된 것들은 기억에 오래 남고 내것으로 남는것 같다.

1. Class 이름과 객체 이름의 충돌

윈도우 visual studio 에서 코딩을 한 후, 80% 이상 했을 때 , 우분투로 옮겨서 동작을 실행시켜 보았다. 그런데 거의 모든 기능이 돌아가던 프로그램이 갑자기 싹다 오류가 나기 시작했다. 문법적으로 전혀 문제가 없었고, 이것저것 고치며 정말 많은 시간을 여기에 투자했었다. 그런데 결과적으로 문제는 객체 이름이었다. 객체 이름을 class 이름과 같이 했더니 자꾸 오류가 생기던 것이었다. 비주얼 스튜디오에서는 자체적으로 보정을 하거나 이를 구분할 줄 알아서 이부분에 대한 오류는 전혀 없었던 것이었다. 그래서 대부분의 코드를 그런식으로 작성한게 화근이었다. 그렇게 모든 객체네임을 바꾸자 정상작동하는것을 확인할 수 있었다. Class 이름과 객체 이름이 겹치면 안되는 이유를 찾아보니 클래스와 객체에 같은 이름을 사용하면, 컴파일러가 이들이 참조하는 대상을 정확히 구분하지 못하게 되어 오류를 발생시킨다고 한다. 따라서 각 클래스와 객체는 고유한 이름을 갖도록 명명하는 것이 중요하다는 것을 이번 기회에 알게 되었다.

2. 명령어 에러코드 처리

두 번째로는 명령어 에러코드를 처리함에 있었다. 그냥 단순히 명령어가 실행 실패했을 때 출력하는 에러코드는 명령어 함수 내에 작성하면 되니 문제가 없었다. 그런데 잘못된 명령어 에러코드 (1000) 가 아닌 인자 개수가 잘못되어 출력하는 에러코드를 구현하는데 위치가 많이 헷갈렸다. 이로 인해서 내가 if-else / while 문의 흐름에 대해 헷갈려 한다는 것을 깨달았다. 나는 그 경우를 모두 설계했다고 생각했는데, 인자 개수 오류일 때 반복문 바깥으로 빠져버려 아무것도 실행되지 않아 시간이 많이 걸렸다. 어려운 알고리즘이 아닌 간단한 반복문에 대한 동작 흐름도 다시 되새기고 눈여겨 볼 필요가 있다는

것을 깨달았다.

3. 그 외

위에서 말한것들은 오래도록 기억에 남을 것 같은 것이였고, 이것들 이외에도 새로 알게된 것들이 있었다, 예를 들면 `using namespace std` 의 정확한 사용과 영향 범위에 대해 아주 디테일 하게 알게 되었다. 이전까지는 그냥 어렴풋이 이름을 혼동하지 않고 사용하기 위해 쓰는 것으로 `std::` 를 쓰는 번거로움만 없다는 것이 장점이다. 라는것밖에는 알지 못했다. 그런데 이번에는 cpp파일과 헤더파일을 직접 코딩하면서 이런 부분까지 자세히 생각해야했다.

그리고 또, 우분투를 처음 사용하면서 신기한 것들을 많이 알게 되었다. 컴퓨터 자체를 공부하고 가까워지는 느낌이었다. cmd명령어 프롬을 사용하면서 경로와 실행에 대해 좀더 생각해 볼 수 있는 기회였다. 이번 프로젝트를 진행하면서 실력이 정말 많이 늘었다고 생각한다. 그리고 단순히 코딩하는 것 뿐만 아니라 이 명령어들과 코드들이 컴퓨터에서 어떻게 작용하는지에 대해 고민해 볼 수 있는 좋은 시간이었다.